

Stat 145 (Multivariate Statistics)

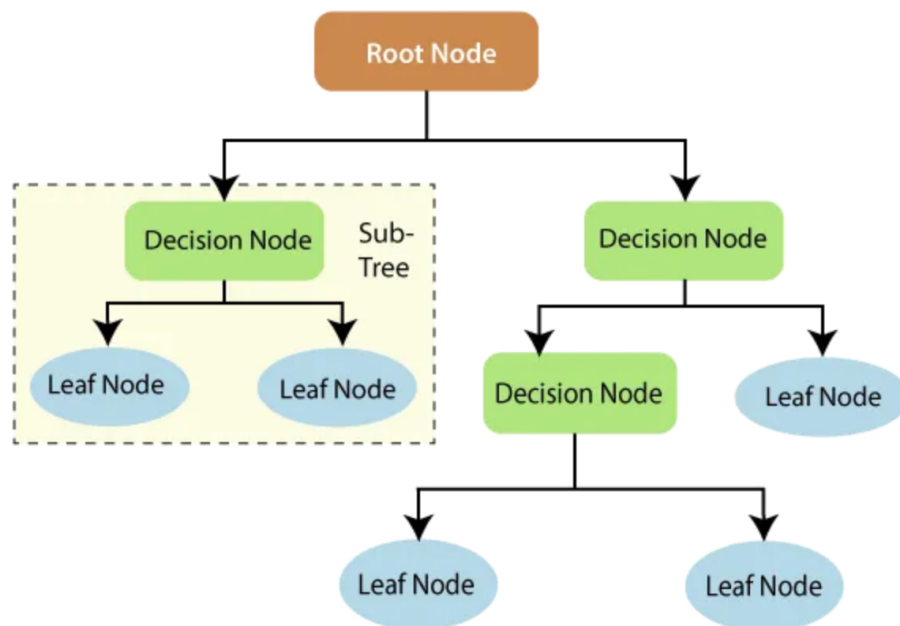
Lesson 3.6 Random Forest Classifier

Understanding decision trees

Decision trees are the building blocks of a random forest algorithm. A decision tree is a decision support technique that forms a tree-like structure. An overview of decision trees will help us understand how random forest algorithms work.

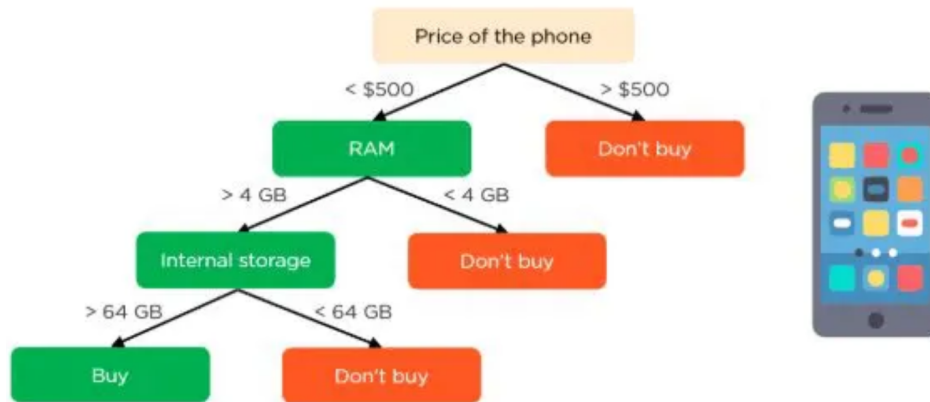
A decision tree consists of three components: decision nodes, leaf nodes, and a root node. A decision tree algorithm divides a training dataset into branches, which further segregate into other branches. This sequence continues until a leaf node is attained. The leaf node cannot be segregated further.

The nodes in the decision tree represent attributes that are used for predicting the outcome. Decision nodes provide a link to the leaves. The following diagram shows the three types of nodes in a decision tree.



Let's take a simple example of how a decision tree works. Suppose we want to predict if a customer will purchase a mobile phone or not. The features of the phone form the basis of his decision. This analysis can be presented in a decision tree diagram.

The root node and decision nodes of the decision represent the features of the phone mentioned above. The leaf node represents the final output, either buying or not buying. The main features that determine the choice include the price, internal storage, and Random Access Memory (RAM). The decision tree will appear as follows.



What is a random forest?

A random forest is a machine learning technique that's used to solve regression and classification problems. It utilizes ensemble learning, which is a technique that combines many classifiers to provide solutions to complex problems.

A random forest algorithm consists of many decision trees. The 'forest' generated by the random forest algorithm is trained through bagging or bootstrap aggregating. Bagging is an ensemble meta-algorithm that improves the accuracy of machine learning algorithms.

The (random forest) algorithm establishes the outcome based on the predictions of the decision trees. It predicts by taking the average or mean of the output from various trees. Increasing the number of trees increases the precision of the outcome.

A random forest eradicates the limitations of a decision tree algorithm. It reduces the overfitting of datasets and increases precision.

Features of a Random Forest Algorithm

1. It's more accurate than the decision tree algorithm.
2. It provides an effective way of handling missing data.
3. It can produce a reasonable prediction without hyper-parameter tuning.
4. It solves the issue of overfitting in decision trees.
5. In every random forest tree, a subset of features is selected randomly at the node's splitting point.

Applying decision trees in random forest

The main difference between the decision tree algorithm and the random forest algorithm is that establishing root nodes and segregating nodes is done randomly in the latter. The random forest employs the bagging method to generate the required prediction.

Bagging involves using different samples of data (training data) rather than just one sample. A training dataset comprises observations and features that are used for making predictions. The decision trees produce different outputs, depending on the training data fed to the random forest algorithm. These outputs will be ranked, and the highest will be selected as the final output.

Our first example can still be used to explain how random forests work. Instead of having a single decision tree, the random forest will have many decision trees. Let's assume we have only four decision trees. In this case, the training data comprising the phone's observations and features will be divided into four root nodes.

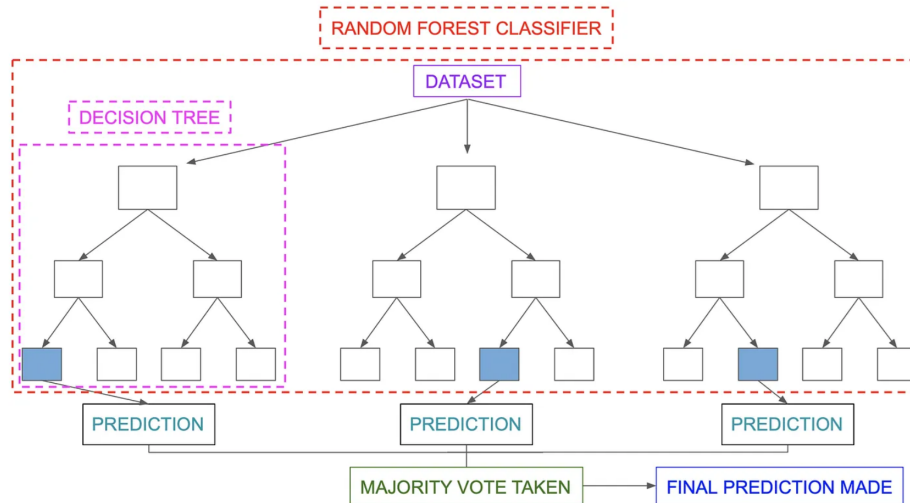
The root nodes could represent four features that could influence the customer's choice (price, internal storage, camera, and RAM). The random forest will split the nodes by selecting features randomly. The final prediction will be selected based on the outcome of the four trees.

The outcome chosen by most decision trees will be the final choice. If three trees predict buying, and one tree predicts not buying, then the final prediction will be buying. In this case, it's predicted that the customer will buy the phone.

Classification in random forests

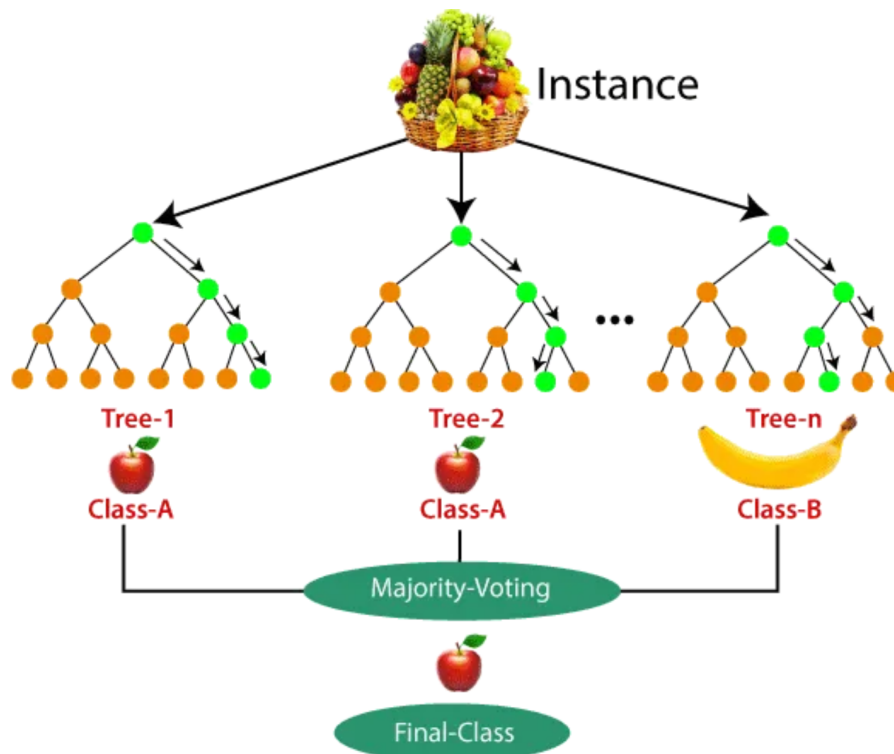
Classification in random forests employs an ensemble methodology to attain the outcome. The training data is fed to train various decision trees. This dataset consists of observations and features that will be selected randomly during the splitting of nodes.

A rain forest system relies on various decision trees. Every decision tree consists of decision nodes, leaf nodes, and a root node. The leaf node of each tree is the final output produced by that specific decision tree. The selection of the final output follows the majority-voting system. In this case, the output chosen by the majority of the decision trees becomes the final output of the rain forest system. The diagram below shows a simple random forest classifier.



Let's take an example of a training dataset consisting of various fruits such as bananas, apples, pineapples, and mangoes. The random forest classifier divides this dataset into subsets. These subsets are given to every decision tree in the random forest system. Each decision tree produces its specific output. For example, the prediction for trees 1 and 2 is apple.

Another decision tree (n) has predicted banana as the outcome. The random forest classifier collects the majority voting to provide the final prediction. The majority of the decision trees have chosen apple as their prediction. This makes the classifier choose apple as the final prediction.



Implementing a random forest classifier in R (Binary outcome)

```
#Loading required packages
library(randomForest)
library(ggplot2)
library(tidyverse)
library(forcats)
library(caret)
library(pROC)
```

The data set

The South African heart disease data can be retrieved from the **catdata** package in R. The dataset has 462 observations with 10 variables. The variables are:

- *y* - coronary heart disease (yes = 1, no = 0)
- *sbp* - systolic blood pressure in millimeters of mercury (mm Hg)
- *tobacco* - cumulative tobacco use in kilograms
- *ldl* - low density lipoprotein cholesterol
- *adiposity* - not recorded in the source, presumably another measurement of obesity similar to BMI. Possibly a “corrected” version of the obesity measure.
- *famhist* - factor indicating the presence or absence of a family history of ischemic heart disease
- *typea* - type-A coronary prone personality behavior as measured by a self-administered Bortner Short Rating Scale. Possible total scores can range from 12 to 84. Rossouw et al. (1983) “arbitrarily” classify those with scores of 55 or more “as exhibiting type A behavior.”
- *obesity* - a measure of obesity; body mass index (or BMI) is consistent with Rossouw et al. (1983). Having a BMI ≥ 30 scored as “obese” by Rossouw et al. (1983).
- *alcohol* - current alcohol consumption
- *age* - age in years at the time of study

```
library(catdata)
data(heart)
heart <- as.data.frame(heart)
```

```
heart <- heart %>%
  mutate(y = factor(y),
         famhist = factor(famhist))
```

```
head(heart)
```

```
##   y sbp tobacco  ldl adiposity famhist typea obesity alcohol age
## 1 1 160   12.00 5.73   23.11      1    49   25.30   97.20  52
## 2 1 144    0.01 4.41   28.61      0    55   28.87    2.06  63
## 3 0 118    0.08 3.48   32.28      1    52   29.14    3.81  46
## 4 1 170    7.50 6.41   38.03      1    51   31.99   24.26  58
## 5 1 134   13.60 3.50   27.78      1    60   25.99   57.34  49
## 6 0 132    6.20 6.47   36.21      1    62   30.77   14.14  45
```

```
#Splitting the data into training and testing sets
```

```
set.seed(1234)
```

```
train_index <- createDataPartition(heart$y, p = 0.7, list = FALSE)
```

```
heart_train <- heart[train_index, ]
```

```
heart_test <- heart[-train_index, ]
```

```
#Displaying the distribution of the outcome (Species) in the training and testing sets
```

```
table(heart_train$y)
```

```
##
##    0    1
## 212 112
```

```
table(heart_test$y)
```

```
##
##    0    1
##  90  48
```

```
#Setting up the 5-fold 2-repeat validation
```

```
control <- trainControl(method = "repeatedcv",
                        number = 5,
                        repeats = 3)
```

```
tuneGrid <- expand.grid(.mtry = c(1:10))
```

```
#Training the model
```

```
set.seed(123)
```

```
heart_rf <- train(y ~ . ,  
                  data = heart_train,  
                  method = "rf",  
                  tuneGrid = tuneGrid,  
                  trControl = control)
```

```
heart_rf$finalModel
```

```
##
```

```
## Call:
```

```
## randomForest(x = x, y = y, mtry = param$mtry)
```

```
##           Type of random forest: classification
```

```
##           Number of trees: 500
```

```
## No. of variables tried at each split: 1
```

```
##
```

```
##           OOB estimate of  error rate: 33.33%
```

```
## Confusion matrix:
```

```
##      0  1 class.error
```

```
## 0 190 22  0.1037736
```

```
## 1  86 26  0.7678571
```

```
#Determining feature importance
```

```
var_imp <- varImp(heart_rf, scale=FALSE)$importance
```

```
var_imp <- data.frame(variables=row.names(var_imp), importance=var_imp$Overall)
```

```
var_imp
```

```
##  variables importance
```

```
## 1      sbp 14.590992
```

```
## 2 tobacco 15.616058
```

```
## 3      ldl 14.691652
```

```
## 4 adiposity 14.096074
```

```
## 5 famhist1  5.920902
```

```
## 6      typea 12.685754
```

```
## 7 obesity 12.834712
```

```
## 8 alcohol 11.197421
```

```
## 9      age 16.138333
```

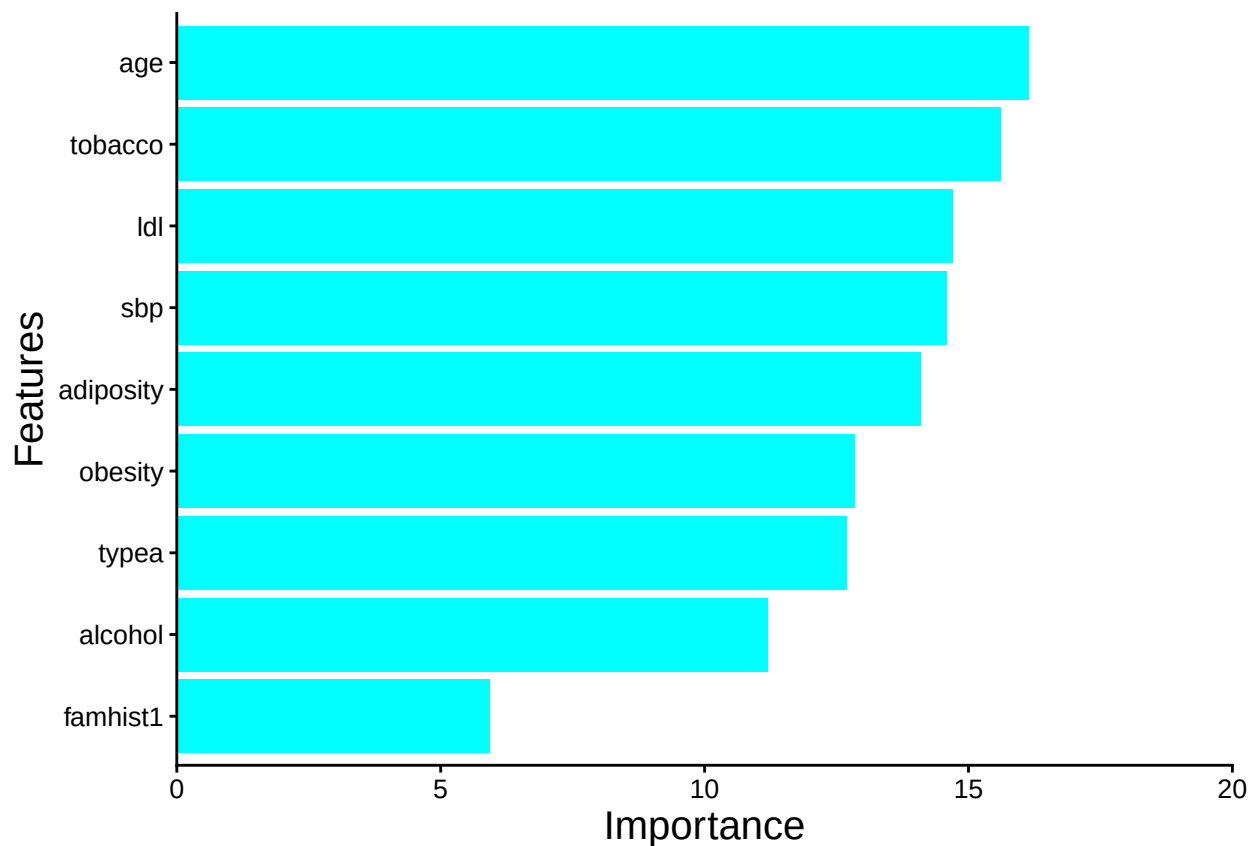
```
## Create a plot of variable importance
```

```
var_imp %>%
```

```
  arrange(importance) %>%
```

```
  ggplot(aes(x=reorder(variables, importance), y=importance)) +
```

```
geom_bar(stat='identity',
         fill = "#00FFFF") +
coord_flip() +
labs(x = 'Features',
     y = "Importance") +
scale_y_continuous(expand = c(0,0),
                   limits = c(0,20))+
theme_classic() +
theme(axis.text = element_text(size = 10),
      axis.title = element_text(size = 15),
      plot.title = element_text(size = 20))
```



#Generating predictions using the test set

```
heart_rf_pred <- predict(heart_rf, heart_test[, -1])
heart_rf_pred
```

```
## [1] 1 0 1 1 0 1 0 1 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
## [38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 1 0 0 0 1 0 0 0 0 0 1 0 0 0 0
## [75] 1 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0
## [112] 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
## Levels: 0 1
```



```
#Generate the confusion matrix using the test set
```

```
confusionMatrix(data = heart_rf_pred,  
                 reference = heart_test$y,  
                 mode = "everything")
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction  0   1
```

```
##           0 82 30
```

```
##           1   8 18
```

```
##
```

```
##           Accuracy : 0.7246
```

```
##           95% CI : (0.6422, 0.7972)
```

```
## No Information Rate : 0.6522
```

```
## P-Value [Acc > NIR] : 0.0428913
```

```
##
```

```
##           Kappa : 0.3204
```

```
##
```

```
## McNemar's Test P-Value : 0.0006577
```

```
##
```

```
##           Sensitivity : 0.9111
```

```
##           Specificity : 0.3750
```

```
## Pos Pred Value : 0.7321
```

```
## Neg Pred Value : 0.6923
```

```
##           Precision : 0.7321
```

```
##           Recall : 0.9111
```

```
##           F1 : 0.8119
```

```
##           Prevalence : 0.6522
```

```
## Detection Rate : 0.5942
```

```
## Detection Prevalence : 0.8116
```

```
## Balanced Accuracy : 0.6431
```

```
##
```

```
## 'Positive' Class : 0
```

```
##
```

```
#Plotting the ROC with AUC
```

```
predict_ROC_heart_rf <- predict(heart_rf, heart_test, type = "prob")
```

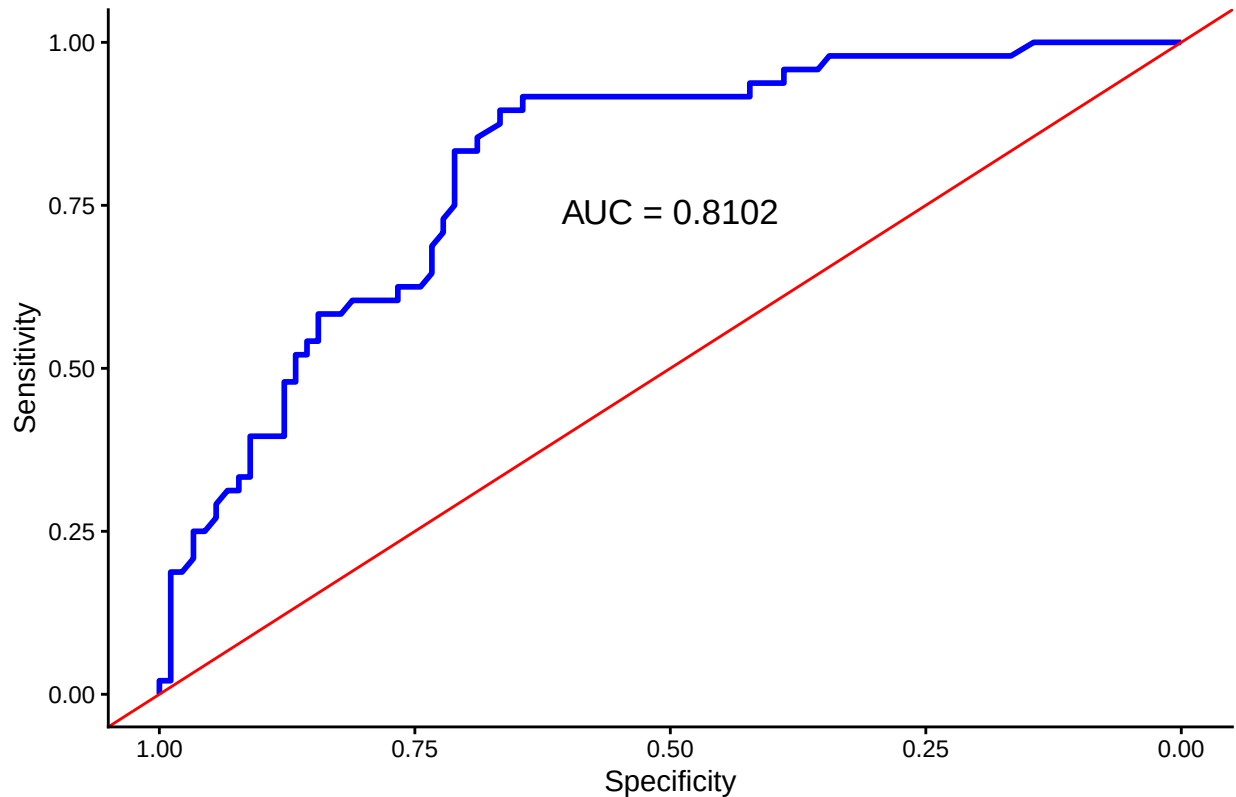
```
ROC_heart_rf <- roc(heart_test$y, predict_ROC_heart_rf[,2])
```

```
ggroc(ROC_heart_rf, col = "blue", size = 1) +  
  ggtitle(paste("AUC =", round(auc(ROC_heart_rf),4))) +  
  labs(x = "Specificity",
```

```

y = "Sensitivity") +
geom_abline(slope = 1, intercept = 1, color = "red")+
theme_classic() +
theme(plot.title = element_text(hjust = 0.5, vjust = -30))

```



Implementing a random forest classifier in R (Multiclass outcome)

```

#Loading the data
data(iris)
head(iris)

```

```

##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5         1.4         0.2   setosa
## 2         4.9         3.0         1.4         0.2   setosa
## 3         4.7         3.2         1.3         0.2   setosa
## 4         4.6         3.1         1.5         0.2   setosa
## 5         5.0         3.6         1.4         0.2   setosa
## 6         5.4         3.9         1.7         0.4   setosa

```

```
#Splitting the data into training and testing sets
set.seed(1234)
train_index1 <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

iris_train <- iris[train_index1, ]
iris_test <- iris[-train_index1, ]
```

```
#Displaying the distribution of the outcome (Species) in the training and testing sets
table(iris_train$Species)
```

```
##
##      setosa versicolor  virginica
##      35          35          35
```

```
table(iris_test$Species)
```

```
##
##      setosa versicolor  virginica
##      15          15          15
```

```
#Setting up the 5-fold 2-repeat validation
control1 <- trainControl(method = "repeatedcv",
                        number = 5,
                        repeats = 3)

tuneGrid1 <- expand.grid(.mtry = c(1:10))
```

```
#Training the model
set.seed(123)
iris_rf <- train(Species ~ .,
                data = iris_train,
                method = "rf",
                tuneGrid = tuneGrid1,
                trControl = control1)

iris_rf$finalModel
```

```
##
## Call:
## randomForest(x = x, y = y, mtry = param$mtry)
##              Type of random forest: classification
##              Number of trees: 500
```

```
## No. of variables tried at each split: 3
##
##          OOB estimate of  error rate: 2.86%
## Confusion matrix:
##          setosa versicolor virginica class.error
## setosa      35          0          0 0.00000000
## versicolor   0          34          1 0.02857143
## virginica    0          2          33 0.05714286
```

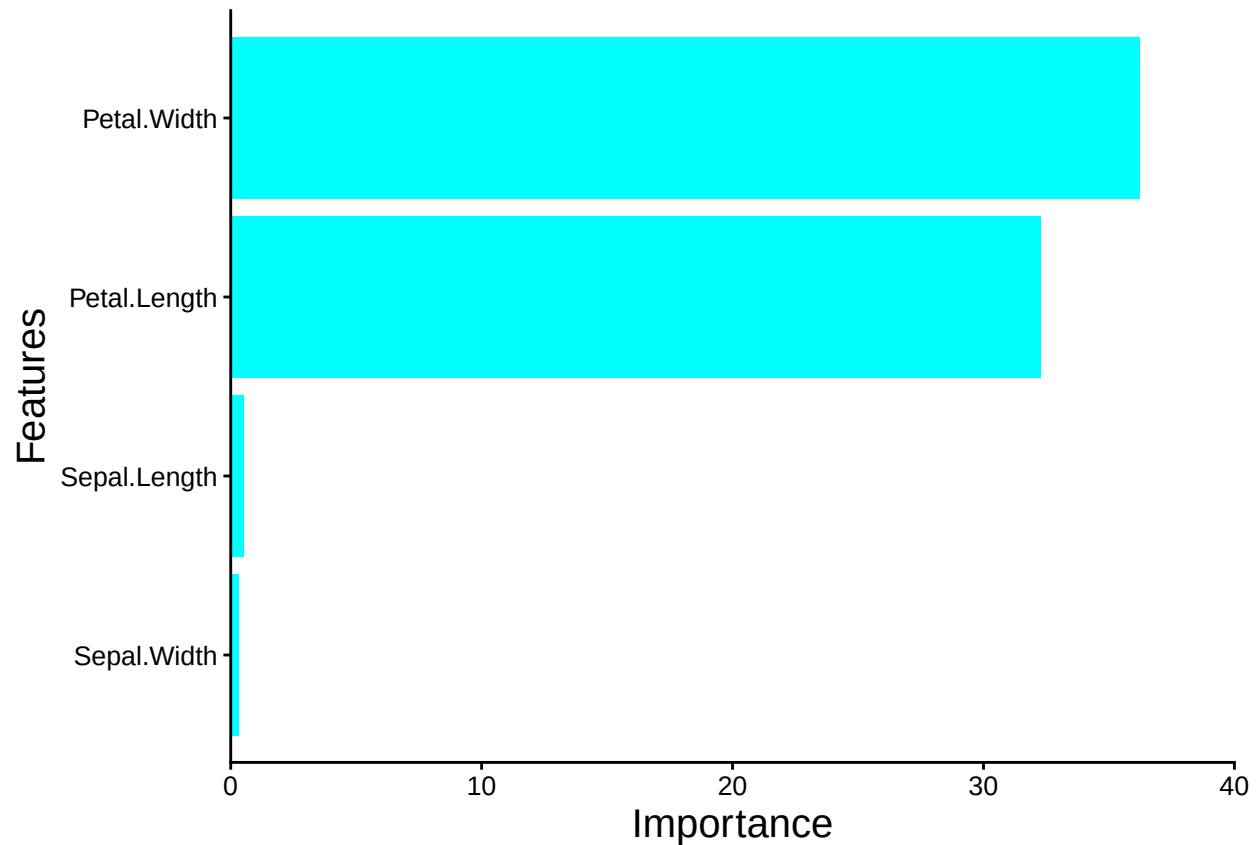
#Determining feature importance

```
var_imp1 <- varImp(iris_rf, scale=FALSE)$importance
var_imp1 <- data.frame(variables=row.names(var_imp1), importance=var_imp1$Overall)
var_imp1
```

```
##          variables importance
## 1 Sepal.Length  0.5134540
## 2  Sepal.Width  0.3076221
## 3 Petal.Length 32.2963518
## 4  Petal.Width 36.2274103
```

Create a plot of variable importance

```
var_imp1 %>%
  arrange(importance) %>%
  ggplot(aes(x=reorder(variables, importance), y=importance)) +
  geom_bar(stat='identity',
           fill = "#00FFFF") +
  coord_flip() +
  labs(x = 'Features',
       y = "Importance") +
  scale_y_continuous(expand = c(0,0),
                     limits = c(0,40))+
  theme_classic() +
  theme(axis.text = element_text(size = 10),
        axis.title = element_text(size = 15),
        plot.title = element_text(size = 20))
```



```
#Generating predictions using the test set
iris_rf_pred <- predict(iris_rf, iris_test[, -5])
```

```
#Generate the confusion matrix using the test set
confusionMatrix(data = iris_rf_pred,
                 reference = iris_test$Species,
                 mode = "everything")
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  setosa versicolor virginica
##   setosa      15          0          0
## versicolor    0          12         0
##  virginica    0           3        15
##
## Overall Statistics
##
##              Accuracy : 0.9333
##              95% CI : (0.8173, 0.986)
```

```
##      No Information Rate : 0.3333
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: setosa Class: versicolor Class: virginica
## Sensitivity              1.0000              0.8000              1.0000
## Specificity              1.0000              1.0000              0.9000
## Pos Pred Value           1.0000              1.0000              0.8333
## Neg Pred Value           1.0000              0.9091              1.0000
## Precision                1.0000              1.0000              0.8333
## Recall                   1.0000              0.8000              1.0000
## F1                       1.0000              0.8889              0.9091
## Prevalence               0.3333              0.3333              0.3333
## Detection Rate           0.3333              0.2667              0.3333
## Detection Prevalence     0.3333              0.2667              0.4000
## Balanced Accuracy         1.0000              0.9000              0.9500
```

#Computing the AUC

```
predict_ROC_iris_rf <- predict(iris_rf, iris_test, type = "prob")
```

```
ROC_iris_rf <- multiclass.roc(iris_test$Species, predict_ROC_iris_rf[,2])
ROC_iris_rf
```

```
##
```

```
## Call:
```

```
## multiclass.roc.default(response = iris_test$Species, predictor = predict_ROC_iris_rf[,2])
```

```
##
```

```
## Data: predict_ROC_iris_rf[, 2] with 3 levels of iris_test$Species: setosa, versicolor, virginica
```

```
## Multi-class area under the curve: 0.8844
```